EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 11-BN: Binary Ninja Personal — Hack Structs & Functions with the NEC IR Remote (Raw `.bin`)

ARM Cortex-M33 • Binary Ninja • OpenOCD

October 12, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the two lesson projects with `Release` and get an `.elf` and a raw `.bin` for each
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load each raw `.bin` into Binary Ninja at `0x10000000`
- **Break at** `main` on live silicon, even though `main` can move between programs
- **Hack each running target live** by editing a register and redirecting a string in Binary Ninja
- **Resolve the functions in the Binary Ninja GUI** using the ELF symbol map
- See how the compiler **flattens a C struct into hard-coded immediates** and **inlines every helper function**
- **Patch** the LED pin immediates and the NEC format string, export, convert, and flash
- Understand the security lesson: **the log says one thing while the hardware does another**

How This Guide Works

Each project builds two files:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table and DWARF	Ground truth for every function address and signature
`.bin`	The raw flash image, no headers, no symbols	The image we load into Binary Ninja and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the current `Release` builds. `Release` flattens the LED struct into plain immediates and inlines every `static` helper (`make_default_leds`, `init_led_gpios`, `process_ir_key`, `poll_ir`, and in Project 2 `ir_to_led_number`, `get_led_pin`, `leds_all_off`, `blink_led`, `process_ir_led_command`, `handle_ir_key`, `poll_and_handle_ir`) into `main`. If you build `Debug`, the SDK function addresses move and the helpers stay separate, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**, twice — once per project:

1. Break on the live target and prove what the code does.
2. Hack it live in the debugger and watch the behavior change.
3. Resolve the functions in Binary Ninja using the ELF symbol map.
4. Patch the bytes, export, convert, and flash.

Project	Prints	Also does	The hacks
`0x0023_structures`	`IR receiver on GPIO 5 ready`, then `NEC command: 0xNN` per key	lights LED1/2/3 on GPIO 16/17/18 from the flattened struct	move LED1 to GPIO 18 live; swap LED1↔LED3 pins; rename the `NEC` string
`0x0026_functions`	the same, plus `LED N activated on GPIO P`	blinks the mapped LED 3× then holds it	forge the decoded key live; swap LED1↔LED3 pins (log desync); rename the `NEC` string

> **The struct disappears.** `simple_led_ctrl_t` has six members (three `uint8_t` pins and three `bool` states), but the optimizer proves it never escapes `main`, so it is never placed in memory. `leds.led1_pin` becomes the literal `16`, `leds.led2_pin` becomes `17`, `leds.led3_pin` becomes `18`, and the `bool` states become register values. That is why you patch **immediates**, not a struct field.

> **The functions disappear too.** Every `static` helper is inlined, so there is no `process_ir_key` or `blink_led` symbol to rename. You see their bodies directly inside `main`. The only real functions `main` calls are the SDK routines and `ir_init` / `ir_getkey`.

> **The SVD file lives in `WEEK04`.** If you want the RP2350 peripheral register map for the SIO/GPIO side of the lab, it is `Embedded-Hacking/WEEK04/rp2350.svd` — it is **not** in `WEEK11`.

Background: the NEC IR remote in one paragraph

An IR receiver on **GPIO 5** demodulates a 38 kHz carrier and presents the NEC frame as a digital mark/space train. `ir_getkey()` waits for the 9 ms leader + 4.5 ms space, samples 32 bits by timing the marks, then validates that the address and command pairs are bitwise inverses. It returns the **command byte** (`0x0C`, `0x18`, or `0x5E` for buttons 1, 2, 3) or `-1`. `main` maps that byte to one of three LEDs on **GPIO 16 (red), 17 (green), 18 (yellow)**. Because the struct is flattened, that mapping is a set of hard-coded pin numbers in the loop — exactly what we patch.

Part 1: Build, Flash, and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **Binary Ninja Personal** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

```
brew install cmake ninja
```

- Install **Binary Ninja Personal** and complete its license activation.
- Install the **Arm GNU Toolchain**, or let the VS Code Pico extension manage it.

Linux x64

```
sudo apt install cmake ninja-build gcc-arm-none-eabi libnewlib-arm-none-eabi git python3 openocd
minicom
```

- Install **Binary Ninja Personal** and complete its license activation.

Step 2: Verify your tools are the right architecture (do not skip this)

On **macOS Apple Silicon**, the most common failure is an Intel `x86_64` tool on your `PATH`:

```
zsh: bad CPU type in executable: cmake
```

You may have **two Homebrews**: the arm64 one at `/opt/homebrew` and the Intel one at `/usr/local`. If `/usr/local/bin` wins, every `brew` tool is x86_64. Check:

```
file "$(which cmake)"
file "$(which ninja)"
file "$(which arm-none-eabi-gdb)"
file "$(which arm-none-eabi-nm)"
file "$(which openocd)"
```

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

To make it permanent, add that `export` to `~/.zshrc`. Do not use Rosetta as a fix; OpenOCD and GDB are exactly the kind of programs where a translation layer produces failures that look like debugger bugs.

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the two projects with `Release`

Run this once inside `0x0023_structures/` and once inside `0x0026_functions/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point Binary Ninja at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so Binary Ninja never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the Binary Ninja console, so the whole build -> patch -> flash loop stays inside Binary Ninja. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
for name in ("0x0023_structures", "0x0026_functions"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
for name in ("0x0023_structures", "0x0026_functions"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
for name in ("0x0023_structures", "0x0026_functions"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Each build directory now contains the pair we need:

- `0x0023_structures/build/0x0023_structures.elf` and `.bin` — `.bin` is **16372** bytes (`0x3ff4`)
- `0x0026_functions/build/0x0026_functions.elf` and `.bin` — `.bin` is **16476** bytes (`0x405c`)

If the ARM toolchain is not on your `PATH` , add `-DPICO_TOOLCHAIN_PATH=...` :

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin`
Linux x64	`/usr`

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on each ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x0023_structures.elf | grep -E ' [Tt] '
arm-none-eabi-nm -n --defined-only build/0x0026_functions.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x0023_structures.elf | Select-String ' [Tt] '
arm-none-eabi-nm -n --defined-only build\0x0026_functions.elf | Select-String ' [Tt] '
```

Each line is `address type name` . The `T` / `t` type is a function. The signatures below come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"` , so they are exact.

Project 1 — `0x0023_structures` — our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init` , `main` , `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (struct flattened, all helpers inlined)
`0x100002cc`	`ir_init`	`void ir_init(uint8_t)`	the `ir.c` receiver init
`0x100002f4`	`ir_getkey`	`int ir_getkey(void)`	the blocking NEC decoder (timing helpers inlined)

Project 1 — the GPIO, UART, stdio, and `printf` chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x100004e8`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select
`0x10000524`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`	SDK pull-up/down (used by `ir_init`)
`0x1000054c`	`gpio_init`	`void gpio_init(uint)`	SDK GPIO init
`0x10000fa8`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay
`0x1000118c`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock (NEC timing)
`0x100011a0`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x10001220`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x100013f4`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART clock lookup
`0x1000310c`	`exit`	`void exit(int)`	C runtime exit
`0x10003114`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10003140`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x10003250`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x1000333c`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x10003364`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x100033f4`	`__wrap_puts`	`int __wrap_puts(const char*)`	the `puts` wrapper (adjacent stdio family)
`0x10003430`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x100034f4`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper (both prints)
`0x100036b0`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init

Address	ELF symbol	Signature	Role
`0x100037f0`	`strlen`	`size_t strlen(const char*)`	C runtime string length

Project 2 — `0x0026_functions` — our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (struct flattened, all helpers inlined)
`0x10000318`	`ir_init`	`void ir_init(uint8_t)`	the `ir.c` receiver init
`0x10000340`	`ir_getkey`	`int ir_getkey(void)`	the blocking NEC decoder (timing helpers inlined)

Project 2 — the GPIO, UART, stdio, and `printf` chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x10000534`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select
`0x10000570`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`	SDK pull-up/down (used by `ir_init`)
`0x10000598`	`gpio_init`	`void gpio_init(uint)`	SDK GPIO init
`0x10000ff0`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay
`0x100011d4`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock (NEC timing)
`0x100011e8`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x10001268`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x1000143c`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART clock lookup
`0x10003154`	`exit`	`void exit(int)`	C runtime exit

Address	ELF symbol	Signature	Role
`0x1000315c`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10003188`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x10003298`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x10003384`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x100033ac`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x1000343c`	`__wrap_puts`	`int __wrap_puts(const char*)`	the `puts` wrapper (adjacent stdio family)
`0x10003478`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x1000353c`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper (all three prints)
`0x100036f8`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x10003838`	`strlen`	`size_t strlen(const char*)`	C runtime string length

> **`main` is `0x10000234` in both projects.** Both programs put `main` at the same address because the startup code and the linker layout are identical; only the body of `main` and the functions after it move. In a `Debug` build the helpers stay separate and `main` moves — another reason to build `Release`.

> **`printf` in our source is `__wrap_printf` in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper, which forwards to `__wrap_vprintf`. The `__wrap_puts` symbol exists in the image (the stdio family always does), but our `printf` path does not call it.

Step 5: Flash Project 1 and confirm the NEC/LED behavior

A `.bin` has no headers, so OpenOCD must be told the base address `0x10000000`. From the repository root:

macOS Apple Silicon / Linux x64:

```
./flash.sh 0x0023_structures/build/0x0023_structures.bin
```

Windows x64 (PowerShell):

```
.\flash.ps1 -Bin 0x0023_structures\build\0x0023_structures.bin
```

Or flash from the Binary Ninja console:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0023_structures", "build", "0x0023_structures.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0023_structures", "build", "0x0023_structures.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 16372 bytes ...` and `** Verified OK **`. Open a serial monitor at **115200** baud:

- **Windows x64:** PuTTY -> Connection type **Serial**, the Pico's COM port, speed `115200`.
- **macOS Apple Silicon:** `screen /dev/tty.usbmodem* 115200` (quit with `Ctrl-A` then `K`).
- **Linux x64:** `minicom -D /dev/ttyACM0 -b 115200`.

On reset it prints the banner once, then a line for every button press:

```
IR receiver on GPIO 5 ready
NEC command: 0x0C      <- button "1" -> red LED (GPIO 16)
NEC command: 0x18      <- button "2" -> green LED (GPIO 17)
NEC command: 0x5E      <- button "3" -> yellow LED (GPIO 18)
```

Step 6: Flash Project 2 and confirm the blink behavior

```
# macOS / Linux
./flash.sh 0x0026_functions/build/0x0026_functions.bin
```

```
# Windows
.\flash.ps1 -Bin 0x0026_functions\build\0x0026_functions.bin
```

Or flash from the Binary Ninja console (same form, pointing at the Project 2 `.bin`):

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0026_functions", "build", "0x0026_functions.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0026_functions", "build", "0x0026_functions.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 16476 bytes ...`. The serial monitor shows the banner, then for each button:

```
IR receiver on GPIO 5 ready
NEC command: 0x0C
LED 1 activated on GPIO 16      <- red LED blinks 3x, then holds on
NEC command: 0x18
LED 2 activated on GPIO 17      <- green LED blinks 3x, then holds on
NEC command: 0x5E
LED 3 activated on GPIO 18      <- yellow LED blinks 3x, then holds on
```

> **Watch the two lines together.** The `LED N activated on GPIO P` line is built from the struct's *pin constants*, which the compiler hard-coded. Later we change the pin the loop actually drives but leave the print constant alone — that is the **log desynchronization** this week is about.

Part 2: Load the Raw `.bin` into Binary Ninja (Project 1)

Start from a fresh Binary Ninja state. If you already have a `.bndb` for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into Binary Ninja

A raw `.bin` has no headers, so Binary Ninja cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, Binary Ninja may load it at address `0x0` with a guessed architecture, and every address in this lesson will be wrong.

1. Choose `File -> Open with Options...` (do **not** use plain `File -> Open`).
2. Select `0x0023_structures/build/0x0023_structures.bin`.
3. In the loader options, set:
   - **Architecture:** `thumb2` (the ARMv7-M / ARMv8-M Thumb-2 architecture, which covers the Cortex-M33)
   - **Platform:** `thumb2`
   - **Base Address:** `0x10000000` (the XIP flash base)
4. Click **Open**.

Binary Ninja analyzes the image and opens the linear view.

Verify the load before going further. Press `G`, type `0x10000000`, and read the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you instead see data at `0x00000000`, or a vector word without bit 0 set, close the tab and repeat with `Open with Options`. The Cortex-M33 only executes Thumb-2, so `thumb2` is the only correct architecture.

> **Console equivalent:**
>
> ```
> load("0x0023_structures/build/0x0023_structures.bin",
>      options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Step 8: Save it as a Binary Ninja database (`.bndb`)

Binary Ninja never writes back into the `.bin`. Your names, comments, types, and patches live in a separate `.bndb` database. Save one now, before you make any changes:

1. Choose `File -> Save As...`.
2. Save it next to the image as `0x0023_structures.bndb`.
3. From now on, save with `File -> Save` (`Cmd+S` on macOS, `Ctrl+S` on Windows/Linux) whenever you rename or patch.

File	Role
`0x0023_structures.bin`	the raw firmware image; Binary Ninja never modifies it
`0x0023_structures.bndb`	your analysis database: names, types, comments, and patches

When you come back later, **open the** `.bndb`, not the `.bin`; that restores all your work. You export the patched image out of this view later, in Step 19.

Step 9: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment. Breakpoints are set from the GUI through the GDB MI adapter — see Step 13.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Dynamic — Break at `main` and Hack Live (Project 1)

Step 10: Start OpenOCD as a live debug server

Make sure no other OpenOCD is running; a forgotten server holds port `3333`.

macOS / Linux:

```
ps aux | grep -i openocd
```

Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -like '*openocd*' }
```

Stop any leftover server gracefully:

macOS / Linux:

```
pkill -TERM -f openocd
```

Windows (PowerShell):

```
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Start the server **parked at** `main`:

macOS Apple Silicon / Linux x64:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Windows x64 (PowerShell):

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Or start it from the Binary Ninja console, freeing the probe first and launching the server in the background so the console returns immediately:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                      os.path.join(root, "debug-server.ps1")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

`Popen` returns in a few milliseconds; the server keeps running in the background. Check `openocd.log` for `Listening on port 3333`, then connect in Step 11.

Wait for:

```
Info : [rp2350.dap.core0] Examination succeed
Startup breakpoint at 0x10000234 (2-byte hardware execute, one-shot).
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

> `BP_ADDR` **parks the core at** `main` **before any client connects.** The script arms a 2-byte hardware breakpoint and then does the startup `reset run`, so the core runs from the vector table and stops at your address with no debugger attached yet. When Binary Ninja connects a moment later, the first thing it reads is already the truth: `Stopped at 0x10000234`.

> **This startup stop is single-use.** OpenOCD flushes breakpoints when a client connects, so this one is gone once Binary Ninja attaches — fine for `main`, which only runs once per reset. Every breakpoint after that is set from the Binary Ninja GUI (Step 13) and is repeatable. To stop at `main` again, restart the server with `BP_ADDR` and reconnect.

> **Exactly one core.** The line must say `core0` and must **not** mention `core1`. Core1 is never started by this firmware; exposing it makes Binary Ninja read core1's reset-state registers, which are not real addresses, and OpenOCD floods the log with `Failed to read memory at 0xf0000000`. The scripts already use `USE_CORE=0`; do not change it.

> **Windows driver note:** the Debug Probe must use the **WinUSB** driver. If OpenOCD reports `unable to open CMSIS-DAP device`, install it with Zadig (select `Debug Probe (CMSIS-DAP)` -> WinUSB).

Step 11: Connect Binary Ninja to the GDB server

1. Make sure the image is open and analyzed (Part 2) and the server from Step 10 is running (parked at `main`).
2. Choose `Debugger -> Connect to Remote Process`.
3. In the **adapter** dropdown, select **GDB MI**.
4. In the **connect** settings group, set **IP Address** to `127.0.0.1` and **Port** to `3333`.
5. Set **Full GDB Executable Path** to the `arm-none-eabi-gdb` from the **Arm GNU Toolchain 14.2.rel1**. It ships for all three hosts, and the Raspberry Pi Pico VS Code extension installs that same 14.2.rel1 toolchain (including `arm-none-eabi-gdb`) on all of them:

OS	`arm-none-eabi-gdb` path
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb` (or the Pico extension's `~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb`)
Windows x64	`%USERPROFILE%\.pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe` (Pico extension), or `C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.2 rel1\bin\arm-none-eabi-gdb.exe`
Linux x64	`~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb` (Pico extension), or the `bin/` directory of the extracted `arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi` tarball

6. Click **Accept**.
7. Restrict the GDB memory map (prevents OpenOCD SWD faults and reconnect loops): In the bottom console panel, click the **Debugger Console** tab (or `View -> Tabs -> Debugger Console`) and enter:

```
source pico-gdb-memory.gdb
```

(If not running from the repository root, enter the absolute path to `pico-gdb-memory.gdb`*.)* Confirm `info mem` displays the Flash (`0x10000000 0x10400000 ro`) and SRAM (`0x20000000 0x20082000 rw`) ranges.

> **Why `source pico-gdb-memory.gdb` is required.** Binary Ninja's register widget automatically previews memory pointed to by registers. When a register contains an unmapped address (such as uninitialized data or constants like `0x69000000`), Binary Ninja probes it. Without a bounded memory map, OpenOCD attempts to read that invalid address over SWD, faults, and reconnects the debug port (`Info : SWD DPIDR 0x4c013477`). Sourcing `pico-gdb-memory.gdb` sets `inaccessible-by-default on` in GDB, causing GDB to intercept and reject invalid reads locally before any packet reaches OpenOCD, eliminating SWD reconnect storms.

> **Use the GDB MI adapter.** It launches a real `arm-none-eabi-gdb --interpreter=mi2` and lets Binary Ninja drive it, so breakpoints and stepping go through real GDB — which sends the correct 2-byte breakpoint length. Verified working end to end: connect, GUI breakpoints (**Add Hardware Breakpoint...**, hardware execute), **Step Into** / **Step Over**, and register edits.

> **Do NOT have any breakpoints set in Binary Ninja before you connect.** With the GDB MI adapter, attaching while Binary Ninja already has a breakpoint **hangs the session**. Start the server parked with `BP_ADDR` (Step 10), connect, and only add hardware breakpoints *after* the connection is up. This is a Binary Ninja bug; it is the single most common GDB MI failure.

> **The GDB executable path matters.** Use the **14.2.rel1** build on every OS (Windows, macOS, Linux). The 13.3.rel1 build did **not** connect in testing.
>
> **This step is temporary.** Vector35 plans to ship a GDB binary with the GDB MI adapter (Vector35/debugger#929, milestone *Langara*). Once that lands, Binary Ninja provides GDB itself and you will not need to set **Full GDB Executable Path** at all.

> **Do not pick Corellium.** Binary Ninja's adapter dropdown also lists **Corellium**, which is for Corellium's virtual devices and expects an API token, not a local OpenOCD server. Always read the label back and confirm it says **GDB MI** before clicking **Accept**.

> **The adapter and port are not saved in the `.bndb`.** Every time you relaunch Binary Ninja you must re-select **GDB MI**, re-enter port `3333`, and re-set the GDB path.

The target keeps running. Open the **Registers** tab (bug icon) and confirm you see live values. `pc` inside `0x10003xxx` and `sp` just below `0x20082000` are healthy.

> **If `pc` is `0x00000088`, `0x000000ec`, or `sp` is `0xf0000000`, the session is bad.** Restart the server, then restart Binary Ninja (a server restart while attached leaves Binary Ninja in a stale session), and connect again.

Step 12: Find `main` without relying on its address

`main` can move between programs, so we do not guess it. We follow the one fixed path to it. Press `G` and go to `0x10000000`:

```
0x10000000   0x20082000   initial stack pointer (top of SRAM)
0x10000004   0x1000015d   reset vector
```

Bit 0 of a vector is the Thumb bit, so `0x1000015d` means "start at `0x1000015c`". That is `_reset_handler`. Follow the reset path to `0x10000186`, `platform_entry`:

```
10000186 <platform_entry>:
10000186:       4914            ldr     r1, [pc, #80]
10000188:       4788            blx     r1
1000018a:       4914            ldr     r1, [pc, #80]
1000018c:       4788            blx     r1
1000018e:       4914            ldr     r1, [pc, #80]
10000190:       4788            blx     r1
10000192:       be00            bkpt    0x0000
```

The middle `blx` at `0x1000018c` is the call to `main`. `platform_entry` is byte-identical in both projects, so `0x1000018c` catches `main` no matter where the linker placed it. The literal pool at `0x100001dc` holds `main | 1`; clearing bit 0 gives `0x10000234`.

Step 13: Set a hardware breakpoint in the GUI

With the **GDB MI** adapter, Binary Ninja sets breakpoints through real GDB, which sends the correct 2-byte length, so you set them **in the UI**. There is no command port here.

Where you can stop

You want to stop at	Project 1 address	How	Repeatable?
`main`	`0x10000234`	The server starts parked there with `BP_ADDR=0x10000234` (Step 10), so Binary Ninja is already stopped at `main` when it connects.	No — `main` runs once per reset.
First `printf` (`IR receiver...`)	`0x1000026c`	Set a hardware breakpoint in the GUI, then click **Resume**.	No — runs once before the loop.
Loop `printf` (`NEC command...`)	`0x1000027c`	Same.	Yes — fires on every key press.
First `gpio_put` (`mcrr`)	`0x1000028a`	Same — this is where LED1's pin is written.	Yes — fires on every key press.
`ir_getkey` **return**	`0x10000274`	Same.	Yes.

Set a breakpoint in the GUI

1. Press `G`, type the address (for example `0x1000028a`), and press Enter.
2. Set a **hardware execution** breakpoint at that address, either way:
   - `Debugger -> Add Hardware Breakpoint...` — a **hardware execute** (`HE`) breakpoint. **Use this one.**
   - click the line and press `F2` (`Debugger -> Toggle Breakpoint`) — a **software** breakpoint. It will **not** work here: the code is in read-only flash, so GDB cannot install it and the core just keeps running.
3. Click **Resume**. The core is already running the loop, so the breakpoint fires on the next key press. Binary Ninja stops with the PC at the address and reports it as a **Breakpoint**.

> **No breakpoints before you connect.** With GDB MI, a breakpoint set before the connection hangs the session (Step 11). Start parked with `BP_ADDR`, connect, *then* add breakpoints.

> **Step Over on the raw `.bin` steps *into* calls.** The raw image has no symbol for `__wrap_printf`, `ir_getkey`, or `sleep_ms`, so **Step Over** at a `bl` behaves like **Step Into**. When the lab needs to execute the call and then stop, it moves the breakpoint to the return site and clicks **Resume** instead (Step 14 shows this).

> **Never use Binary Ninja's Restart button.** On RP2350 it resets and halts inside the boot ROM (`pc=0x88`, `sp=0xf0000000`). To reset cleanly, restart the server with `BP_ADDR` and reconnect.

Step 14: HACK IT LIVE — move LED1 from GPIO 16 to GPIO 18

`main` loads the constant `16` into `r5` once at `0x10000240`, before the loop, and the loop's first `mcrr` at `0x1000028a` writes the LED1 state to the pin in `r5`. Because `r5` is **never reloaded inside the loop**, changing it once is sticky for every later key press. We break on that `mcrr` and change it live.

1. Press `G`, go to `0x1000028a` (the first `mcrr`, `gpio_put(r5, ...)` for LED1).
2. Set a **hardware execute** breakpoint there: `Debugger -> Add Hardware Breakpoint...`. (Do not use `F2` — that is a software breakpoint and will not work on read-only flash.)
3. Click **Resume** in Binary Ninja, then press **"1"** on the IR remote. The `ir_getkey` call returns, the `printf` at `0x1000027c` prints `NEC command: 0x0C`, and the breakpoint fires at `0x1000028a`.
4. Open the **Registers** widget (bug icon -> **Registers**). Find `r5`. Its value is `16` (`0x10`) — LED1's pin.
5. **Set `r5` to `18` (`0x12`).** From Binary Ninja's Python console (`Plugins -> Python Console`):

   ```
   dbg.set_reg_value("r5", 0x12)  # LED1 now drives GPIO 18
   ```

   `dbg.set_reg_value(name, value)` writes one register (returns `True` on success). You can also right-click `r5` in the **Registers** widget, press `E` (edit), type `12`, and press Enter. The widget may not repaint the value, but the write reaches the target.
6. Remove the breakpoint at `0x1000028a` and click **Resume**. The core executes the `mcrr` with `r5 = 18`, so pressing **"1"** now lights the **yellow** LED on GPIO 18 instead of the red LED on GPIO 16.

Because `r5` is set once and never reloaded, the change sticks for every subsequent key press until you reset. Press **"1"** again: the yellow LED lights again, while the terminal still says `NEC command: 0x0C`.

> `r5` **is the sticky register here.** If you instead edit `r3` (the state, computed by the `clz` trick) or `r2`, the next iteration recomputes them, so the edit lasts one pass. `r5` is the pin, loaded once, so it is the one worth moving.

Step 14b: HACK THE STRING LIVE — change `NEC` to `HACKED`

The text `"NEC command: 0x%02X\n"` lives in flash (`.rodata`) at `0x100038d0`, and flash is **read-only at runtime** — a debugger write there does not stick. So you cannot overwrite the text in place. Instead you redirect the pointer: at the loop `printf` call, `r0` holds the format-string address, so you point `r0` at a replacement string you place in RAM.

1. Press `G`, go to `0x1000027c` (the loop `bl __wrap_printf`) and set a **hardware execute** breakpoint. Resume and press **"1"** on the remote. At the stop, `r0 = 0x100038d0` (the `ldr r0, [pc, #76]` at `0x1000027a` just loaded the `"NEC command: 0x%02X\n"` pointer) and `r1 = 0x0C`.
2. Put the replacement string into free RAM at `0x20080000` from the **Python console**:

   ```
   dbg.write_memory(0x20080000, b"HACKED: 0x%02X\n\x00")  # one %02X, same argument
   ```

   `dbg.write_memory(address, bytes)` is Binary Ninja's debugger memory-write API; it returns `True` on success. Keep exactly one `%02X` so `printf` still consumes the key in `r1`.
3. Point `r0` at that string:

   ```
   dbg.set_reg_value("r0", 0x20080000)
   ```

   (Or right-click `r0` in the **Registers** widget, press `E`, type `20080000`, and press Enter.)
4. Move the breakpoint past the call in the GUI (remove it at `0x1000027c`, set one at `0x10000280`) and click **Resume**. The core runs `printf` with `r0` pointing at your RAM string and `r1 = 0x0C`, so this iteration prints:

   ```
   HACKED: 0x0C
   ```

   then stops at `0x10000280`.

Like the pin hack, this is **one iteration only**: the loop reloads `r0` from the literal pool on every pass, so the next key prints `NEC command: ...` again. The permanent version is the static patch in Step 18b.

Step 15: Why the hack reverts (and why we patch next)

Press **Resume**. The loop branches back to `0x10000270`, reloads `r0` from `0x100038d0` at `0x1000027a`, and `r5` stays at `18` only until the next reset (it is loaded once at `0x10000240`). The string edit was one iteration; the pin edit was sticky but lives only in a register. To make the behavior permanent we must patch the bytes — the static pass.

Press **Pause** to stop the output flood.

Step 15b: Kill the debugger and OpenOCD

The live hack is done. Do this **before** the static pass.

1. In the **Debugger** sidebar, click the **X** (**Kill**) (or `Debugger -> Kill`) to disconnect Binary Ninja.
2. **Kill does not stop the OpenOCD process** — `debug-server.sh` started it separately, and it keeps running and holding the probe. Stop it from the Binary Ninja console:

   macOS / Linux:

   ```
   import subprocess
   subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
   ```

   Windows:

   ```
   import subprocess
   subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the
   probe
   ```

3. Confirm nothing is left: `pgrep -fl openocd` (macOS/Linux) prints nothing.

From a terminal it is the same: `pkill -TERM -f openocd`, or `Get-Process openocd | Stop-Process` on Windows.

Part 4: Static — Resolve the Functions in Binary Ninja and Patch (Project 1)

Step 16: Resolve the functions in the Binary Ninja GUI

We now name the functions in Binary Ninja using the ELF symbol map from Step 4. Binary Ninja loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	Binary Ninja action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in Binary Ninja (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_10003364()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. Binary Ninja parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter.

> **Binary Ninja shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and Binary Ninja's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `ir_init`

1. `G` -> `0x100002cc`.
2. `Y` -> `void ir_init(uint8_t pin)`.

It takes a `uint8_t` pin; `main` calls it with `5` (`movs r0, #5`). It opens with `gpio_init(pin)`, then writes the direction and calls `gpio_set_pulls` for the pull-up.

Worked example: `ir_getkey`

1. `G` -> `0x100002f4`.
2. `Y` -> `int ir_getkey(void)`.

It returns `-1` on timeout and the command byte otherwise. The NEC timing helpers (`wait_for_level`, `wait_leader`, `read_nec_bit`, `read_32_bits`, `validate_nec_frame`) are all `static` and **inlined** into it, so you will not find them as separate functions.

Worked example: `gpio_init`

1. `G` -> `0x1000054c`.
2. `Y` -> `void gpio_init(uint gpio)`.

`main` calls it three times with `16`, `17`, `18` — the flattened struct pins.

Worked example: `stdio_init_all`

1. `G` -> `0x10003364`.
2. `Y` -> `bool stdio_init_all(void)`.

It returns `bool`, not `void` — the ELF says `_Bool stdio_init_all(void)`. Our `main` ignores the return value, so the decompiler still reads cleanly.

Worked example: `__wrap_printf`

1. `G` -> `0x100034f4`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

> **`printf` in our source is `__wrap_printf` in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper.

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK. The call chain for this project:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init
│   │                                    └── uart_init ── clock_get_hz, busy_wait_us
│   ├── stdio_set_driver_enabled
│   ├── stdio_out_chars_crlf
│   └── stdio_put_string ── strlen, time_us_64
├── gpio_init
├── ir_init ── gpio_init, gpio_set_pulls
├── ir_getkey ── time_us_64   (the NEC timing helpers are inlined)
└── __wrap_printf ── __wrap_vprintf
```

Project 1 — resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`
`0x100002cc`	`ir_init`	`void ir_init(uint8_t)`
`0x100002f4`	`ir_getkey`	`int ir_getkey(void)`
`0x100004e8`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`
`0x10000524`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`
`0x1000054c`	`gpio_init`	`void gpio_init(uint)`
`0x10000fa8`	`sleep_ms`	`void sleep_ms(uint32_t)`
`0x1000118c`	`time_us_64`	`uint64_t time_us_64(void)`
`0x100011a0`	`busy_wait_us`	`void busy_wait_us(uint64_t)`
`0x10001220`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`
`0x100013f4`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`
`0x1000310c`	`exit`	`void exit(int)`

Address	Rename to (N)	Signature (Y)
`0x10003114`	`runtime_init`	`void runtime_init(void)`
`0x10003140`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`
`0x10003250`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`
`0x1000333c`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`
`0x10003364`	`stdio_init_all`	`bool stdio_init_all(void)`
`0x100033f4`	`__wrap_puts`	`int __wrap_puts(const char*)`
`0x10003430`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`
`0x100034f4`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`
`0x100036b0`	`stdio_uart_init`	`void stdio_uart_init(void)`
`0x100037f0`	`strlen`	`size_t strlen(const char*)`

> **A `void` return type may not stick — here is the fix.** Binary Ninja treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type Binary Ninja thinks it can infer.
>
> Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:
>
> ```
> from binaryninja import ReturnValue, Type
> fn = bv.get_function_at(0x1000015c)
> if fn is not None:
>     fn.return_value = ReturnValue(Type.void())
> ```
>
> That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into Binary Ninja's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x100002cc: ("ir_init",                   "void ir_init(uint8_t)"),
    0x100002f4: ("ir_getkey",                 "int ir_getkey(void)"),
    0x100004e8: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x10000524: ("gpio_set_pulls",            "void gpio_set_pulls(uint, bool, bool)"),
    0x1000054c: ("gpio_init",                 "void gpio_init(uint)"),
    0x10000fa8: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x1000118c: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x100011a0: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x10001220: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x100013f4: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x1000310c: ("exit",                      "void exit(int)"),
    0x10003114: ("runtime_init",              "void runtime_init(void)"),
    0x10003140: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10003250: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x1000333c: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10003364: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x100033f4: ("__wrap_puts",               "int __wrap_puts(const char*)"),
    0x10003430: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x100034f4: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x100036b0: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x100037f0: ("strlen",                    "size_t strlen(const char*)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)
```

> SDK type names (`stdio_driver_t` , `uart_inst_t` , `gpio_function_t` , `clock_handle_t` , plus `uint` , `va_list`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first. Standard names (`uint8_t` , `uint32_t` , `uint64_t` , `bool` , `size_t`) are built in.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main` . The whole program is one function because every `static` helper was inlined:

```
10000234 <main>:
10000234:       b508            push    {r3, lr}
10000236:       f003 f895       bl      10003364 <stdio_init_all>
1000023a:       2010            movs    r0, #16
1000023c:       f000 f986       bl      1000054c <gpio_init>
10000240:       2510            movs    r5, #16
10000242:       f04f 0401       mov.w   r4, #1
10000246:       ec44 5044       mcrr    0, 4, r5, r4, cr4
1000024a:       2011            movs    r0, #17
1000024c:       f000 f97e       bl      1000054c <gpio_init>
10000250:       2311            movs    r3, #17
10000252:       ec44 3044       mcrr    0, 4, r3, r4, cr4
10000256:       2012            movs    r0, #18
10000258:       f000 f978       bl      1000054c <gpio_init>
1000025c:       2312            movs    r3, #18
1000025e:       ec44 3044       mcrr    0, 4, r3, r4, cr4
10000262:       2005            movs    r0, #5
10000264:       f000 f832       bl      100002cc <ir_init>
10000268:       2105            movs    r1, #5
1000026a:       4816            ldr     r0, [pc, #88]
1000026c:       f003 f942       bl      100034f4 <__wrap_printf>
10000270:       f000 f840       bl      100002f4 <ir_getkey>
10000274:       1e04            subs    r4, r0, #0
10000276:       db21            blt.n   100002bc <main+0x88>
10000278:       4621            mov     r1, r4
1000027a:       4813            ldr     r0, [pc, #76]
1000027c:       f003 f93a       bl      100034f4 <__wrap_printf>
10000280:       f1a4 030c       sub.w   r3, r4, #12
10000284:       fab3 f383       clz     r3, r3
10000288:       095b            lsrs    r3, r3, #5
1000028a:       ec43 5040       mcrr    0, 4, r5, r3, cr0
1000028e:       f1a4 0218       sub.w   r2, r4, #24
10000292:       fab2 f282       clz     r2, r2
10000296:       2311            movs    r3, #17
10000298:       0952            lsrs    r2, r2, #5
1000029a:       ec42 3040       mcrr    0, 4, r3, r2, cr0
1000029e:       f1a4 045e       sub.w   r4, r4, #94
100002a2:       fab4 f484       clz     r4, r4
100002a6:       2312            movs    r3, #18
100002a8:       0964            lsrs    r4, r4, #5
100002aa:       ec44 3040       mcrr    0, 4, r3, r4, cr0
100002ae:       200a            movs    r0, #10
100002b0:       f000 fe7a       bl      10000fa8 <sleep_ms>
100002b4:       f000 f81e       bl      100002f4 <ir_getkey>
100002b8:       1e04            subs    r4, r0, #0
100002ba:       dadd            bge.n   10000278 <main+0x44>
100002bc:       2001            movs    r0, #1
100002be:       f000 fe73       bl      10000fa8 <sleep_ms>
100002c2:       e7d5            b.n     10000270 <main+0x3c>
100002c4:       100038b0        .word   0x100038b0
100002c8:       100038d0        .word   0x100038d0
```

The decompiler reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    gpio_init(0x10); gpio_set_dir(0x10, 1);   // led1_pin = 16
    gpio_init(0x11); gpio_set_dir(0x11, 1);   // led2_pin = 17
    gpio_init(0x12); gpio_set_dir(0x12, 1);   // led3_pin = 18
    ir_init(5);
    __wrap_printf("IR receiver on GPIO %d ready\n", 5);
    do
    {
        int32_t key = ir_getkey();
        if (key >= 0)
        {
            __wrap_printf("NEC command: 0x%02X\n", key);
            mcrr(0x10, key == 0x0c);   // led1_pin = 16
            mcrr(0x11, key == 0x18);   // led2_pin = 17
            mcrr(0x12, key == 0x5e);   // led3_pin = 18
            sleep_ms(10);
        }
        else
        {
            sleep_ms(1);
        }
    } while (true);
}
```

- **There is no struct.** `16`, `17`, `18` are immediates; `led1_state` etc. are the `clz`-computed register values.
- The `clz` / `lsrs` pair is how the compiler turns `(key == 0x0C)` into a `0` / `1` without a branch: `sub.w r3, r4, #12` sets the flags, `clz r3, r3` counts leading zeros, `lsrs r3, r3, #5` reduces it to `0` or `1`.
- `0x100002c4` and `0x100002c8` point at `"IR receiver on GPIO %d ready\n"` (`0x100038b0`) and `"NEC command: 0x%02X\n"` (`0x100038d0`) in `.rodata`.

Step 18: Patch 1 — swap LED1 and LED3 pins

The original lesson swaps LED pin assignments. LED1 is the red LED on GPIO 16 and LED3 is the yellow LED on GPIO 18. Swap them so button **"1"** lights yellow and button **"3"** lights red. The two pins are hard-coded immediates:

Address	Instruction	Bytes before	Bytes after	Role
`0x10000240`	`movs r5, #16`	`10 25`	`12 25`	LED1's pin (`r5`) 16 -> 18
`0x100002a6`	`movs r3, #18`	`12 23`	`10 23`	LED3's pin (`r3`) 18 -> 16

In the **Hex** view (`View -> Hex`, lock off) change the low byte of each, then right-click `main` -> `Reanalyze`. Or in the Python console:

```
bv.write(0x10000240, b"\x12")  # movs r5, #18  (LED1 -> GPIO 18)
bv.write(0x100002a6, b"\x10")  # movs r3, #16  (LED3 -> GPIO 16)
print(bv.read(0x10000240, 2).hex(" "))  # -> 12 25
print(bv.read(0x100002a6, 2).hex(" "))  # -> 10 23
```

After reanalysis the loop reads `movs r5, #18` and `movs r3, #16`, so button **"1"** drives GPIO 18 (yellow) and button **"3"** drives GPIO 16 (red). The `NEC command:` log still prints the *command byte*, so the log and the physical LED mapping no longer agree — the log desynchronization.

Step 18b: Patch 2 — rename the `NEC` string to `PWN`

The format string `"NEC command: 0x%02X\n"` starts at `0x100038d0`. Its first three bytes are `4e 45 43` (`NEC`). Change them to `50 57 4e` (`PWN`), leaving the `command: 0x%02X\n` tail untouched, so the line prints `PWN command: 0x0C`.

Option A — Hex view: go to `0x100038d0` and change the three bytes `4e 45 43` to `50 57 4e`, then reanalyze.

Option B — Python console:

```
bv.write(0x100038d0, b"PWN")
print(bv.read(0x100038d0, 20))  # -> b'PWN command: 0x%02X\n\x00'
```

Keep the replacement exactly three bytes. If you use a shorter string you must pad it, or `%02X` shifts and `printf` reads the wrong argument. A longer string would overwrite the `command:` tail.

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x0023_structures", "build"), "0x0023_structures-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 16372 /.../build/0x0023_structures-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base Binary Ninja loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x3ff4` = 16372). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.

> **No relative path.** Binary Ninja's Python console runs with a read-only working directory (inside the app bundle), so a relative `open(...)` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/`.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0023_structures-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0023_structures-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the Binary Ninja console** — `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x0023_structures", "build"))  # the project build dir (writable)
> sys.argv = ["uf2conv.py", "0x0023_structures-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")   # path to your uf2conv.py
> ```

Step 21: Flash and verify the swapped LEDs

Hold **BOOTSEL**, plug in the Pico 2, and drag `hacked.uf2` onto the `RP2350` drive. Or flash the `.bin` over the Debug Probe with SWD — no BOOTSEL — from the console (stop any running OpenOCD first, and use `Popen`, not `run`, so the console is not blocked):

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
bin_path = os.path.join(os.path.join(root, "0x0023_structures", "build"), "0x0023_structures-
h.bin")
log = os.path.join(os.path.join(root, "0x0023_structures", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Open the serial monitor and press the buttons:

```
PWN command: 0x0C      <- button "1" now lights the YELLOW LED (GPIO 18)
PWN command: 0x18      <- button "2" still lights the green LED (GPIO 17)
PWN command: 0x5E      <- button "3" now lights the RED LED (GPIO 16)
```

Two bytes swapped the LEDs and three bytes renamed the log — no source code.

Part 5: Reflash Project 2 and Load It into Binary Ninja

Step 22: Reflash Project 2 and restart the session

Part 4 left the Pico running the patched Project 1 image. Put the original Project 2 back and start a fresh session.

1. Stop any running debug server so the flash script can use the probe:

```
# macOS / Linux
pkill -TERM -f openocd
```

```
# Windows
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

2. Flash the original Project 2 image:

```
# macOS / Linux
./flash.sh 0x0026_functions/build/0x0026_functions.bin
```

```
# Windows
.\flash.ps1 -Bin 0x0026_functions\build\0x0026_functions.bin
```

Or do steps 1–2 from the Binary Ninja console:

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step
3)
bin_path = os.path.join(root, "0x0026_functions", "build", "0x0026_functions.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing Project 2 in the background; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step
3)
bin_path = os.path.join(root, "0x0026_functions", "build", "0x0026_functions.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing Project 2 in the background; log:", log)
```

3. Load Project 2 and save its database — see Step 22b.

Confirm the Pico responds to `1` / `2` / `3` again.

Step 22b: Load Project 2 into Binary Ninja and save the database

Exactly like Steps 7–8, but for Project 2. **Use** `File -> Open with Options...` (not plain `File -> Open`), select `0x0026_functions/build/0x0026_functions.bin`, and set:

- **Architecture:** `thumb2`
- **Platform:** `thumb2`
- **Base Address:** `0x10000000`

Click **Open**. Then press `G`, type `0x10000000`, and confirm the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you see data at `0x00000000`, close the tab and redo it with `Open with Options`.

Save it with `File -> Save As...` as `0x0026_functions.bndb` (next to the `.bin`). From now on open the `.bndb`, not the `.bin`; save with `Cmd+S` / `Ctrl+S` after every rename or patch.

> **Console equivalent:**
>
> ```
> load("0x0026_functions/build/0x0026_functions.bin",
>      options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Part 6: Dynamic — Break at `main` and Hack Live (Project 2)

Step 23: Break at `main`

`main` is at `0x10000234` in this project too. Start the server parked at `main` (Step 10 form) and connect with the **GDB MI** adapter (Step 11):

1. Restart the server parked at `main`:

   macOS / Linux:

   ```
   BP_ADDR=0x10000234 ./debug-server.sh
   ```

   ```
   # Windows
   $env:BP_ADDR="0x10000234"; .\debug-server.ps1
   ```

   Or restart it from the Binary Ninja console:

   macOS / Linux:

   ```
   import os, subprocess
   root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
   log = os.path.join(root, "openocd.log")
   subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # kill any running server first
   subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                    env=dict(os.environ, BP_ADDR="0x10000234"),
                    stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
   print("OpenOCD restarted parked at main; log:", log)
   ```

   Windows:

   ```
   import os, subprocess
   root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
   log = os.path.join(root, "openocd.log")
   subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # kill any running server first
   subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                     os.path.join(root, "debug-server.ps1")], cwd=root,
                    env=dict(os.environ, BP_ADDR="0x10000234"),
                    stdout=open(log, "w"), stderr=subprocess.STDOUT)
   print("OpenOCD restarted parked at main; log:", log)
   ```

2. Connect Binary Ninja (Step 11): adapter **GDB MI**, IP `127.0.0.1`, port `3333`.

The target is already halted at `main` when Binary Ninja connects, and the sidebar reads `Stopped at 0x10000234`.

Step 24: Read `main` and find the inlined function bodies

The whole loop is one function because every helper was inlined. The key difference from Project 1 is the extra `printf` and the blink loop that carries the LED pin in `r5`:

```
10000234 <main>:
10000234:       b580            push    {r7, lr}
10000236:       f003 f8b9       bl      100033ac <stdio_init_all>
1000023a:       2010            movs    r0, #16
1000023c:       f000 f9ac       bl      10000598 <gpio_init>
10000240:       f04f 0701       mov.w   r7, #1
10000244:       2310            movs    r3, #16
10000246:       ec47 3044       mcrr    0, 4, r3, r7, cr4
1000024a:       2011            movs    r0, #17
1000024c:       f000 f9a4       bl      10000598 <gpio_init>
10000250:       2311            movs    r3, #17
10000252:       ec47 3044       mcrr    0, 4, r3, r7, cr4
10000256:       2012            movs    r0, #18
10000258:       f000 f99e       bl      10000598 <gpio_init>
1000025c:       2312            movs    r3, #18
1000025e:       ec47 3044       mcrr    0, 4, r3, r7, cr4
10000262:       2005            movs    r0, #5
10000264:       f000 f858       bl      10000318 <ir_init>
10000268:       2105            movs    r1, #5
1000026a:       4828            ldr     r0, [pc, #160]
1000026c:       f003 f966       bl      1000353c <__wrap_printf>
10000270:       f04f 0600       mov.w   r6, #0
10000274:       f000 f864       bl      10000340 <ir_getkey>
10000278:       1e04            subs    r4, r0, #0
1000027a:       db19            blt.n   100002b0 <main+0x7c>
1000027c:       4621            mov     r1, r4
1000027e:       4824            ldr     r0, [pc, #144]
10000280:       f003 f95c       bl      1000353c <__wrap_printf>
10000284:       2510            movs    r5, #16
10000286:       ec46 5040       mcrr    0, 4, r5, r6, cr0
1000028a:       2311            movs    r3, #17
1000028c:       ec46 3040       mcrr    0, 4, r3, r6, cr0
10000290:       2212            movs    r2, #18
10000292:       ec46 2040       mcrr    0, 4, r2, r6, cr0
10000296:       2c0c            cmp     r4, #12
10000298:       d00e            beq.n   100002b8 <main+0x84>
1000029a:       2c18            cmp     r4, #24
1000029c:       d02e            beq.n   100002fc <main+0xc8>
1000029e:       2c5e            cmp     r4, #94
100002a0:       d030            beq.n   10000304 <main+0xd0>
100002a2:       200a            movs    r0, #10
100002a4:       f000 fea4       bl      10000ff0 <sleep_ms>
100002a8:       f000 f84a       bl      10000340 <ir_getkey>
100002ac:       1e04            subs    r4, r0, #0
100002ae:       dae5            bge.n   1000027c <main+0x48>
100002b0:       2001            movs    r0, #1
100002b2:       f000 fe9d       bl      10000ff0 <sleep_ms>
100002b6:       e7dd            b.n     10000274 <main+0x40>
100002b8:       f04f 0801       mov.w   r8, #1
100002bc:       2403            movs    r4, #3
100002be:       ec47 5040       mcrr    0, 4, r5, r7, cr0
100002c2:       2032            movs    r0, #50
100002c4:       f000 fe94       bl      10000ff0 <sleep_ms>
100002c8:       ec46 5040       mcrr    0, 4, r5, r6, cr0
100002cc:       2032            movs    r0, #50
100002ce:       f000 fe8f       bl      10000ff0 <sleep_ms>
100002d2:       1e63            subs    r3, r4, #1
100002d4:       f013 04ff       ands.w  r4, r3, #255
100002d8:       d1f1            bne.n   100002be <main+0x8a>
100002da:       ec47 5040       mcrr    0, 4, r5, r7, cr0
100002de:       f1b8 0f01       cmp.w   r8, #1
100002e2:       d009            beq.n   100002f8 <main+0xc4>
100002e4:       f1b8 0f02       cmp.w   r8, #2
100002e8:       bf14            ite     ne
100002ea:       2212            movne   r2, #18
100002ec:       2211            moveq   r2, #17
100002ee:       4641            mov     r1, r8
100002f0:       4808            ldr     r0, [pc, #32]
100002f2:       f003 f923       bl      1000353c <__wrap_printf>
```

```
100002f6:       e7d4            b.n     100002a2 <main+0x6e>
100002f8:       2210            movs    r2, #16
100002fa:       e7f8            b.n     100002ee <main+0xba>
100002fc:       461d            mov     r5, r3
100002fe:       f04f 0802       mov.w   r8, #2
10000302:       e7db            b.n     100002bc <main+0x88>
10000304:       4615            mov     r5, r2
10000306:       f04f 0803       mov.w   r8, #3
1000030a:       e7d7            b.n     100002bc <main+0x88>
1000030c:       100038f8        .word   0x100038f8
10000310:       10003918        .word   0x10003918
10000314:       10003930        .word   0x10003930
```

What the inlining produced:

- `leds_all_off(&leds)` is the three `mcrr` writes at `0x10000286`, `0x1000028c`, `0x10000292`, all using `r6 = 0` (`mov.w r6, #0` at `0x10000270`).
- `ir_to_led_number` is the `cmp` / `beq` chain at `0x10000296` – `0x100002a0` (`12`, `24`, `94`).
- `get_led_pin` is the `mov r5, r3` at `0x100002fc` (key 24 -> pin 17) and the `mov r5, r2` at `0x10000304` (key 94 -> pin 18); for key 12, `r5` keeps the `16` loaded at `0x10000284`.
- `blink_led` is the loop at `0x100002be` – `0x100002d8`; `r4` counts 3 down to 0, `r5` is the pin, `r7 = 1` and `r6 = 0` drive it on/off.
- `get_led_pin` **in the final print** is `movs r2, #16` at `0x100002f8` (key 1), `moveq r2, #17` at `0x100002ec` (key 2), and `movne r2, #18` at `0x100002ea` (key 3). These are the constants that will **lie** after we patch the loop pins.

The string map, read straight from `.rodata`:

Literal	Points at	String
`0x1000030c`	`0x100038f8`	`"IR receiver on GPIO %d ready\n"`
`0x10000310`	`0x10003918`	`"NEC command: 0x%02X\n"`
`0x10000314`	`0x10003930`	`"LED %d activated on GPIO %d\n"`

Step 25: HACK IT LIVE — forge the decoded NEC key

`ir_getkey` returns the command byte in `r0`; `main` copies it into `r4` at `0x10000278`. We stop right after the read and overwrite `r4` so the program takes a different button's path — even though the operator pressed a different button.

1. Press `G`, go to `0x1000027c` (the `mov r1, r4` right after the `blt.n`, inside the `key >= 0` block). Set a **hardware execute** breakpoint: `Debugger -> Add Hardware Breakpoint...`.
2. Click **Resume** and press **"1"** on the IR remote. `ir_getkey` returns, `subs r4, r0, #0` at `0x10000278` runs, and the breakpoint fires at `0x1000027c` with `r4 = 0x0C` (12).
3. **Set `r4` to `0x5E` (94)** from the Python console:

   ```
   dbg.set_reg_value("r4", 0x5E)  # pretend button "3" was pressed
   ```

   (Or right-click `r4` in the **Registers** widget, press `E`, type `5e`, and press Enter.)
4. Remove the breakpoint at `0x1000027c` and click **Resume**. The core runs `mov r1, r4`, so the `printf` prints `NEC command: 0x5E`, the `cmp` chain takes the key-94 branch at `0x10000304`, and the Pico blinks the **yellow** LED on GPIO 18 — although you pressed **"1"**.

> **The pin variant.** If you prefer to move the pin instead of the key, break at `0x10000286` (the first `mcrr`, after `movs r5, #16` at `0x10000284`) and set `r5 = 0x12`. LED1's blink then drives GPIO 18, but `r5` is reloaded at `0x10000284` on the next key, so it is a one-key change. The `r4` edit above is the same idea one step earlier in the pipeline.

Step 25b: HACK THE STRING LIVE — change `NEC` to `HACKED`

The `"NEC command: 0x%02X\n"` format is at `0x10003918`; redirect `r0` to a RAM string at the `printf` call.

1. Press `G`, go to `0x10000280` (the `bl __wrap_printf` on the key path) and set a **hardware execute** breakpoint. Resume and press **"1"**. At the stop, `r0 = 0x10003918` — the `ldr r0, [pc, #144]` at `0x1000027e` loaded the pointer — and `r1 = 0x0C`.
2. Write the replacement to RAM and repoint `r0`:

   ```
   dbg.write_memory(0x20080000, b"HACKED: 0x%02X\n\x00")  # keep one %02X
   dbg.set_reg_value("r0", 0x20080000)
   ```

3. Remove the breakpoint at `0x10000280`, set one at `0x10000284`, and click **Resume**. This iteration prints:

   ```
   HACKED: 0x0C
   ```

   One iteration only — the loop reloads `r0` from the literal pool each pass. The permanent version is the static patch in Step 27b.

Step 25c: Kill the debugger and OpenOCD

Same as Step 15b: click the **X** (**Kill**) in the **Debugger** sidebar (or `Debugger -> Kill`), then stop OpenOCD from the Binary Ninja console:

macOS / Linux:

```
import subprocess
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
```

Windows:

```
import subprocess
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the probe
```

Part 7: Static — Resolve the Functions and Patch (Project 2)

Step 26: Resolve the functions in the Binary Ninja GUI

Same two keys as Step 16 — `G` to the address, then `Y` (Change Type) to set the prototype — using the Project 2 ELF symbol map from Step 4.

Worked example: `main`

1. `G` -> `0x10000234`.
2. `Y` -> `int main(void)` (Binary Ninja shows `int32_t main(void)` — the same 32-bit `int`).

Worked example: `ir_init`

1. `G` -> `0x10000318`.
2. `Y` -> `void ir_init(uint8_t pin)`.

Worked example: `ir_getkey`

1. `G` -> `0x10000340`.
2. `Y` -> `int ir_getkey(void)`.

Worked example: `gpio_init`

1. `G` -> `0x10000598`.
2. `Y` -> `void gpio_init(uint gpio)`.

`main` calls it three times with `16`, `17`, `18` — the flattened struct pins.

Worked example: `__wrap_printf`

1. `G` -> `0x1000353c`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic. It forwards to `__wrap_vprintf`.

Worked example: `sleep_ms`

1. `G` -> `0x10000ff0`.
2. `Y` -> `void sleep_ms(uint32_t ms)`.

The blink loop and the idle path both load `10` or `50` immediately before calling it.

The call chain for this project is the same as Project 1, plus the extra `printf` and the blink loop:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init
│   │                                    └── uart_init ── clock_get_hz, busy_wait_us
│   ├── stdio_set_driver_enabled
│   ├── stdio_out_chars_crlf
│   └── stdio_put_string ── strlen, time_us_64
├── gpio_init
├── ir_init ── gpio_init, gpio_set_pulls
├── ir_getkey ── time_us_64   (the NEC timing helpers are inlined)
├── __wrap_printf ── __wrap_vprintf   (NEC line and LED line)
└── sleep_ms
```

Project 2 — resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`

Address	Rename to (N)	Signature (Y)
0x10000318	ir_init	void ir_init(uint8_t)
0x10000340	ir_getkey	int ir_getkey(void)
0x10000534	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x10000570	gpio_set_pulls	void gpio_set_pulls(uint, bool, bool)
0x10000598	gpio_init	void gpio_init(uint)
0x10000ff0	sleep_ms	void sleep_ms(uint32_t)
0x100011d4	time_us_64	uint64_t time_us_64(void)
0x100011e8	busy_wait_us	void busy_wait_us(uint64_t)
0x10001268	uart_init	uint uart_init(uart_inst_t*, uint)
0x1000143c	clock_get_hz	unsigned long clock_get_hz(clock_handle_t)
0x10003154	exit	void exit(int)
0x1000315c	runtime_init	void runtime_init(void)
0x10003188	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10003298	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x10003384	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x100033ac	stdio_init_all	bool stdio_init_all(void)
0x1000343c	__wrap_puts	int __wrap_puts(const char*)
0x10003478	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x1000353c	__wrap_printf	int __wrap_printf(const char*, ...)
0x100036f8	stdio_uart_init	void stdio_uart_init(void)
0x10003838	strlen	size_t strlen(const char*)

Shortcut — resolves name *and* type for every function. Paste this into Binary Ninja's Python console:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x10000318: ("ir_init",                   "void ir_init(uint8_t)"),
    0x10000340: ("ir_getkey",                 "int ir_getkey(void)"),
    0x10000534: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x10000570: ("gpio_set_pulls",            "void gpio_set_pulls(uint, bool, bool)"),
    0x10000598: ("gpio_init",                 "void gpio_init(uint)"),
    0x10000ff0: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x100011d4: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x100011e8: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x10001268: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x1000143c: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x10003154: ("exit",                      "void exit(int)"),
    0x1000315c: ("runtime_init",              "void runtime_init(void)"),
    0x10003188: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10003298: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x10003384: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x100033ac: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x1000343c: ("__wrap_puts",               "int __wrap_puts(const char*)"),
    0x10003478: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x1000353c: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x100036f8: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10003838: ("strlen",                    "size_t strlen(const char*)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)
```

Step 27: Patch 1 — swap LED1 and LED3 pins

The original lesson swaps LED 1 and LED 3. LED1 is the red LED on GPIO 16 and LED3 is the yellow LED on GPIO 18. In the loop the pins are `movs r5, #16` at `0x10000284` (LED1) and `movs r2, #18` at `0x10000290` (LED3). Swap the two immediates:

Address	Instruction	Bytes before	Bytes after	Role
`0x10000284`	`movs r5, #16`	`10 25`	`12 25`	LED1's pin (`r5`) 16 -> 18
`0x10000290`	`movs r2, #18`	`12 22`	`10 22`	LED3's pin (`r2`) 18 -> 16

```
bv.write(0x10000284, b"\x12")  # movs r5, #18  (LED1 -> GPIO 18)
bv.write(0x10000290, b"\x10")  # movs r2, #16  (LED3 -> GPIO 16)
print(bv.read(0x10000284, 2).hex(" "))  # -> 12 25
print(bv.read(0x10000290, 2).hex(" "))  # -> 10 22
```

Now button **"1"** blinks GPIO 18 (yellow) and button **"3"** blinks GPIO 16 (red). The `LED N activated on GPIO P` prints are **not** changed, so they still say `GPIO 16` and `GPIO 18` — **the log no longer matches the hardware.** That mismatch is the security lesson: the operator's console shows the old, expected mapping while the pins do something else.

> **Optional consistency patch.** If you want the print to tell the truth instead, also change `movs r2, #16` at `0x100002f8` to `#18` (and the key-3 print constant `movne r2, #18` at `0x100002ea` to `#16`). For this lesson we leave them alone on purpose, so the desynchronization is visible.
>
> | Address | Instruction | Bytes before | Bytes after | Role |
> |---|---|---|---|---|
> | `0x100002f8` | `movs r2, #16` | `10 22` | `12 22` | optional: key-1 print now says GPIO 18 |

Step 27b: Patch 2 — rename the `NEC` string to `PWN`

The format string starts at `0x10003918` ; change its first three bytes `4e 45 43` (`NEC`) to `50 57 4e` (`PWN`):

```
bv.write(0x10003918, b"PWN")
print(bv.read(0x10003918, 20))  # -> b'PWN command: 0x%02X\n\x00'
```

Exactly three bytes, same rule as Project 1: a shorter string must be padded, a longer one overwrites the `command:` tail.

Step 28: Export, convert, and flash

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size from the view itself
out = os.path.join(os.path.join(root, "0x0026_functions", "build"), "0x0026_functions-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 16476 /.../build/0x0026_functions-h.bin
```

`seg.data_length` is the image size (`0x405c` = 16476) read from the view — nothing hardcoded.

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0026_functions-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0026_functions-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Or run the conversion from the Binary Ninja console, exactly as in Step 20 (`os.chdir` to the build dir, then `runpy.run_path("../../uf2conv.py", run_name="__main__")` with `sys.argv` set to the arguments above).

Hold **BOOTSEL**, plug in the Pico 2, drag `hacked.uf2` onto the `RP2350` drive. Or flash the `.bin` over the Debug Probe with SWD — no BOOTSEL — from the console, exactly as in Step 21.

Step 29: Verify

Open the serial monitor:

- press **"1"** -> the **YELLOW** LED on GPIO 18 blinks (it used to be the red LED on GPIO 16), and the terminal still prints `LED 1 activated on GPIO 16` — **wrong**, it is actually GPIO 18;
- press **"3"** -> the **RED** LED on GPIO 16 blinks (it used to be the yellow LED on GPIO 18), and the terminal still prints `LED 3 activated on GPIO 18` — **wrong**, it is actually GPIO 16;
- press **"2"** -> the green LED on GPIO 17 is unchanged;
- every `NEC command:` line now reads `PWN command:`.

The log says one thing, the hardware does another — with two bytes and no source code.

Cheatsheet

Binary Ninja GUI actions

Action	How
Go to address	`G`
Rename function/symbol	`N`
Set type or signature	`Y`
Add comment	`;`
Open Hex view	`View -> Hex`
Enable hex editing	Toggle the lock in the status bar
Reanalyze after a patch	Right-click function -> `Reanalyze`
Edit a register live	`dbg.set_reg_value("r4", 0x5E)` in the Python console (or right-click the register, press `E`, type hex, Enter)

Action	How
Write debugger memory	`dbg.write_memory(0x20080000, b"HACKED: 0x%02X\n\x00")`
Set a breakpoint	`Debugger -> Add Hardware Breakpoint...` (hardware execute). Do **not** use `F2` — software breakpoints cannot be written to read-only flash.
Move a breakpoint	Remove it and set it at the new address in the GUI (command-port fallback: `rbp <old addr>` then `bp <new addr> 2 hw`)
Confirm what is armed	The **Breakpoints** widget lists it (command-port fallback: `mdw 0xE0002000 8`, each armed breakpoint shows as `<addr | 1>`)
Apply the ELF symbol map	Paste the Python snippet from Step 16 / 26 into the Python Console

OpenOCD server and reset

The server runs with `gdb_breakpoint_override hard` so that flash-writes are never attempted. Breakpoints in this lab are set in the Binary Ninja GUI through the **GDB MI** adapter (Step 13). The command-port rows below are the fallback if you use the **GDB RSP** adapter instead.

Action	Command
Connect to the OpenOCD prompt (fallback)	`nc 127.0.0.1 4444` (or `telnet 127.0.0.1 4444`)
Reset and run (command port)	`reset run`
Check core state (command port)	`targets`
Set a breakpoint in the GUI	`Debugger -> Add Hardware Breakpoint...` (hardware execute; `F2` software breakpoints do not work on flash)
(fallback) Add a breakpoint without the GUI	`bp <addr> 2 hw`
Remove one breakpoint	`rbp <addr>` — **address only, no length, no** `hw`
Remove every breakpoint	`rbp all`
Start the server parked at `main`	macOS/Linux: `BP_ADDR=0x10000234 ./debug-server.sh` — Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1` (one-shot)
Break on the loop in a running target	set a hardware breakpoint in the GUI at the loop address, then **Resume** — repeatable
Make Binary Ninja stepping work	`rp2350.dap.core0 configure -rtos none` (already in the scripts)
Step without re-trapping	move the breakpoint off the current PC first, then **Step Into**/**Step Over**
Reset without desyncing Binary Ninja	**Detach**, `reset run` on the port, reconnect — never `reset run` while attached
Kill the debugger	click **X** in the **Debugger** sidebar, or `Debugger -> Kill`

Action	Command
Stop OpenOCD	macOS/Linux: `pkill -TERM -f openocd` — Windows: `taskkill /F /IM openocd.exe`

Where you can stop

Stop at	Project 1 `0x0023`	Project 2 `0x0026`
`main` (once per reset)	`0x10000234`	`0x10000234`
`ir_getkey` return	`0x10000274`	`0x10000274`
Loop `printf` (`NEC command`)	`0x1000027c`	`0x10000280`
First `gpio_put` (`mcrr`, LED1 pin)	`0x1000028a`	`0x10000286`
LED3 `gpio_put` (`mcrr`)	`0x100002aa`	`0x10000292`

Every address and byte we changed

Project	Address	Before	After	Effect
`0x0023`	`0x10000240`	`10`	`12`	LED1 pin 16 -> 18
`0x0023`	`0x100002a6`	`12`	`10`	LED3 pin 18 -> 16
`0x0023`	`0x100038d0`	`4e 45 43`	`50 57 4e`	prints `PWN` instead of `NEC`
`0x0026`	`0x10000284`	`10`	`12`	LED1 pin 16 -> 18
`0x0026`	`0x10000290`	`12`	`10`	LED3 pin 18 -> 16
`0x0026`	`0x100002f8`	`10`	`12`	optional: key-1 print says GPIO 18
`0x0026`	`0x10003918`	`4e 45 43`	`50 57 4e`	prints `PWN` instead of `NEC`

Raw image facts

Item	Value
Build type	`Release`
Load base address	`0x10000000`
Project 1 size	`16372` bytes (`0x3ff4`)
Project 2 size	`16476` bytes (`0x405c`)
Initial stack pointer (both)	`0x20082000`
Reset vector (both)	`0x1000015d`
Fixed `main` anchor (both)	`0x1000018c`
`main` (both)	`0x10000234`
`ir_init`, Project 1	`0x100002cc`

Item	Value
`ir_init`, Project 2	`0x10000318`
`ir_getkey`, Project 1	`0x100002f4`
`ir_getkey`, Project 2	`0x10000340`
Project 1 `IR receiver` string	`0x100038b0`
Project 1 `NEC command` string	`0x100038d0`
Project 2 `IR receiver` string	`0x100038f8`
Project 2 `NEC command` string	`0x10003918`
Project 2 `LED activated` string	`0x10003930`
RP2350 UF2 family ID	`0xe48bff59`

Troubleshooting

Binary Ninja hangs or crashes when you connect (macOS 27)

Three different causes have been seen on this setup; check them in this order.

- **A breakpoint set before connecting.** With the **GDB MI** adapter, if the binary view already has a breakpoint, the session hangs. Start parked with `BP_ADDR`, connect, then add breakpoints.
- **The wrong GDB executable.** Point **Full GDB Executable Path** at the **14.2.rel1** toolchain (Step 11). The 13.3.rel1 build did not connect in testing.
- **The LLDB adapter.** A crash report with `libdebuggercore.dylib -> std::terminate() -> abort()` and `liblldb` in the stack is the **LLDB** adapter, not GDB MI. Avoid LLDB on this setup.

If Binary Ninja hangs, force-quit it; the connect dialog has no working Cancel. The static steps (resolve, patch, export, flash) never touch the debugger and always work.

GDB MI hangs when you connect (a breakpoint already existed)

With the **GDB MI** adapter, if Binary Ninja already has a breakpoint set when you connect, the session **hangs**. The working order is:

1. Start the server parked, e.g. `BP_ADDR=0x10000234 ./debug-server.sh` (Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1`).
2. Connect with the **GDB MI** adapter.
3. Only *then* set hardware breakpoints in the UI.

Step Into / Step Over does nothing (PC never moves)

Two causes have been seen on this target.

1. **A breakpoint on the current PC re-traps the step.** OpenOCD's step-over-breakpoint logic fails with `Duplicate Breakpoint address` and the PC stays put. Fix: move the breakpoint off the current PC (in the GUI), then step.
2. **The `hwthread` RTOS (GDB RSP adapter only).** With the **GDB RSP** adapter, OpenOCD can log `fake step thread 0` and reply without stepping. Fix: `rp2350.dap.core0 configure -rtos none`. **GDB MI does not hit this.**

`zsh: bad CPU type in executable: cmake`

An Intel `x86_64` tool is on your `PATH` on Apple Silicon. Run Step 2: `export PATH="/opt/homebrew/bin:$PATH"`, then `hash -r`. Add it to `~/.zshrc` to make it permanent.

My addresses do not match this guide

You probably built `Debug`. This lesson is a `Release` build. Re-run Step 3 with `-DCMAKE_BUILD_TYPE=Release`. A `Debug` build moves the SDK functions and keeps the `static` helpers separate, so `main` is not at `0x10000234`.

A breakpoint never fires

First, confirm you actually set one, and that it is a **hardware** breakpoint. `Debugger -> Add Hardware Breakpoint...` (hardware execute) should land in the **Breakpoints** widget. If nothing lands, or the core keeps running, you probably used `F2` (`Toggle Breakpoint`) — a software breakpoint cannot be written to read-only flash. Also check you are on **GDB MI**, not **GDB RSP** (the GDB RSP adapter cannot set breakpoints on this target at all).

Then check the order and the state:

- **Arm it only after Binary Ninja is connected.** OpenOCD flushes every breakpoint when a client attaches, so anything armed earlier is gone. This also applies to `BP_ADDR` on the startup command line.
- **Is the core running?** If it is stopped, click **Resume**.
- **Does the address get reached again?** `main` runs once per reset, so use `BP_ADDR` at startup rather than `reset run` while attached. Loop addresses such as `0x1000028a` (P1) and `0x10000286` (P2) fire on the next key press with no reset. For the `NEC` print addresses (`0x1000027c` / `0x10000280`) you must also press a remote button, because they sit inside the `if (key >= 0)` block.

I edit `r4` / `r5` and it reverts

For Project 2's key forgery, `subs r4, r0, #0` reloads `r4` from `ir_getkey` on every key press, so the edit is visible for one key. For Project 1's `r5` pin edit, `r5` is loaded once at `0x10000240` and never reloaded, so it sticks until reset. For Project 2's `r5` pin edit, `movs r5, #16` at `0x10000284` reloads it on every key. The edit sticks only while the core is **genuinely stopped** at the breakpoint.

> **The Registers widget is a snapshot, not a live view.** Binary Ninja reads the registers at each stop and shows that snapshot; it does not poll the target. A value changed outside Binary Ninja will not appear until the next stop.

The string hack does nothing (or prints garbage)

Pick a RAM address that is free — `0x20080000` is safe here (well above the `.data` / `.bss` end at about `0x20000810`). Write a NUL-terminated string, and keep exactly the format specifiers the call consumes: the NEC format has one `%02X`, so the replacement must keep one `%02X`. Then set `r0`, not `r1`.

The patched string shifted `printf` output

The `NEC command: 0x%02X\n` format has one `%02X`. Keep the replacement exactly three bytes (`NEC` -> `PWN`); a longer string would overwrite the `command:` tail and a shorter one would leave a stray character. In the live hack you write a whole new NUL-terminated string to RAM, so any length is fine as long as it keeps one `%02X`.

The struct is not in memory — where is it?

It is not. `Release` proved `simple_led_ctrl_t` never escapes `main`, so the compiler **flattened** it: the three `uint8_t` pins became the immediates `16`, `17`, `18` and the three `bool` states became register values. There is no `sub sp` for the struct and no memory address to inspect. You patch the immediates instead. If you need to see the struct in memory, build `Debug`, but then none of the addresses in this guide apply.

Project 2's `LED N activated on GPIO P` line is wrong after the patch

That is the point. Step 27 swaps the pins the loop *drives* but leaves the print constants (`0x100002f8`, `0x100002ea`, `0x100002ec`) untouched, so the log shows the old mapping. If you want the print to match, apply the optional consistency patch in Step 27.

The serial capture is garbage on macOS

Reading `/dev/cu.usbmodem*` with a bare `read()` returns garbage. Set **raw termios at 115200** first, or just use `screen /dev/cu.usbmodem* 115200`, which does it for you.

It worked for a second, then stopped (Binary Ninja's view desyncs)

The main cause is **driving the core from the OpenOCD command port while Binary Ninja is connected**. If you must reset, **Detach first**, reset, then reconnect. Never leave a breakpoint on the PC you are about to step or resume from.

The console floods with `Failed to read memory` or `SWD DPIDR` reconnects

Two causes:

1. **Core 1 is exposed.** The scripts must run with `USE_CORE=0`. Stop the server, confirm only `core0` is reported, restart, then restart Binary Ninja.
2. **Binary Ninja probed an unmapped address for a register preview.** When a register holds an arbitrary integer or uninitialized value (e.g. `0x69000000`), Binary Ninja tries to preview that memory address, triggering an OpenOCD SWD fault and debug-port reconnect (`SWD DPIDR 0x4c013477`). In the **Debugger Console**, run `source pico-gdb-memory.gdb`. This restricts GDB to valid Flash and SRAM, filtering invalid reads locally so OpenOCD's SWD connection stays completely stable.

The decompiler still shows the old value after patching

Right-click the function and choose `Reanalyze`.

Fallback: do the dynamic steps with GDB (macOS 27)

If Binary Ninja's debugger crashes on attach on macOS 27, you can still do the live hacks with the ARM GDB from the toolchain, against the same OpenOCD server. The addresses and register values are identical to the GUI steps.

Start the debug server (Step 10), then in a new terminal:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt for Project 1:

```
set architecture armv8-m.main
target extended-remote :3333
hbreak *0x1000028a
continue
```

Do **not** run `monitor reset run` before `hbreak`. `0x1000028a` is inside `main`'s loop, so the breakpoint fires on the next key press with no reset. Press **"1"** on the remote, then:

```
info registers pc r5  # pc = 0x1000028a, r5 = 0x10
set $r5 = 0x12
continue
```

The next LED1 write drives GPIO 18 — the same temporary live hack as editing `r5` in the Binary Ninja Registers widget. For the string hack, break at `0x1000027c`, then `set {char[19]}0x20080000 = "HACKED: 0x%02X\n"` and `set $r0 = 0x20080000`.

Project 2 is the same with the other call site and value:

```
hbreak *0x1000027c
continue
info registers pc r4  # r4 holds the decoded key you pressed
set $r4 = 0x5E
continue
```

`hbreak` sets a hardware breakpoint, which is required for read-only flash. It works from plain GDB because GDB sends the 2-byte length the Cortex-M33 comparators need. Binary Ninja's **GDB MI** adapter goes through the same GDB, so its GUI breakpoints work too.

Glossary

Term	Definition
`.bss`	Section for uninitialized global variables; zeroed by startup code
`.data`	Section for initialized global variables; copied from flash to SRAM at boot
`.elf`	Linked image with the symbol table; the ground truth for addresses and names

Term	Definition
Flattening	The optimizer replacing struct member accesses with the member's constant value; why `simple_led_ctrl_t` disappears
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
Hardware breakpoint	A breakpoint serviced by the CPU comparators, required for read-only flash
Inlining	The optimizer replacing a function call with the function body; why every `static` helper disappears from `main`
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`
`mcrr`	Move to coprocessor from two registers — how the SIO GPIO writes are encoded
NEC	A common IR protocol: 9 ms leader + 4.5 ms space, then 32 data bits (address, ~address, command, ~command)
`.rodata`	Read-only section for constants and string literals; stays in flash
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at `0xd0000000`
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Prove the behavior dynamically, resolve the names from the ELF, then patch the bytes and flash.